TREINAMENTO CYPRESS · SERVEREST

DIA 1

Fundamentos

01

O que vamos fazer hoje

- **1.1** Estratégia de testes e o que é o Cypress
- **1.2** JavaScript essencial
- **1.3** Instalação e configuração
- **1.4** Seu primeiro teste
- **1.5** Comandos e asserções
- **1.6** Hooks e independência
- **1.7** Seletores e debugging
- **1.8** Lab

Ao fim do dia

Um projeto Cypress rodando na sua máquina
e uma suíte de testes escrita por você

1.1

Estratégia e visão geral

Quais testes vale a pena automatizar?

Não são todos

Teste automatizado tem **dois** custos:

Escrever — visível, todo mundo considera

Manter — invisível, e é o que mata suítes

Cada teste que você escreve é um teste que alguém vai precisar consertar quando a tela mudar.

A pirâmide



Poucos testes E2E
caros, lentos, frágeis — mas os únicos que provam o sistema de ponta a ponta

Testes de integração
mais baratos; validam a conversa entre as partes

Muitos testes unitários
baratíssimos e rápidos — mas não provam que o produto funciona

Não é regra moral. É observação sobre **custo**.

Automatize o que dói se quebrar

Não tudo

O que costuma valer

- Fluxos que geram receita
- Fluxos que, se quebrarem, param o negócio
- Caminhos que você já viu quebrar
- Cenários repetitivos e chatos de testar na mão

O que costuma não valer

- Tela que muda toda semana
- Validação puramente visual
- Caso que acontece uma vez por ano

O Cypress roda dentro do navegador

No mesmo processo que a sua aplicação

Por dentro x por fora

Selenium e similares mandam comandos por um protocolo externo.
Como operar um computador por controle remoto.

Cypress está dentro da sala.

Tudo o que vem a seguir é consequência disso.

O que você ganha

	Por quê
Espera automática	Ele sabe quando a página mudou
Acesso ao que está por baixo	`window`, `localStorage`, rede
Velocidade e estabilidade	Sem latência de protocolo
Time Travel	Ver a tela em qualquer passo

O que você perde

Limitação	Na prática
Menos navegadores	Chrome, Edge, Firefox, Electron. Sem Safari real
Múltiplas abas	Não dá para controlar duas ao mesmo tempo
Origem cruzada	Restrições ao navegar entre domínios

Quando não usar: teste de carga, mobile nativo, scraping.

Duas palavras do slide anterior

Origem cruzada

Origem = protocolo + domínio + porta. O Cypress fica preso ao site que abriu.

Sair para outro no meio do teste — login do Google, pagamento no banco — exige `cy.origin()`.

Scraping

Robô que visita sites para **extrair dados**: preços, notícias, tabelas.

O Cypress testa uma aplicação; para scraping há Puppeteer e Playwright.

Demo

Time Travel

Perguntas que sempre aparecem

Cypress substitui o Selenium?

Não. Mesmo problema, trade-offs diferentes.

Preciso saber programar?

JavaScript básico. Não dá para fugir — e é menos do que parece.

E o Playwright?

Concorrente direto e muito capaz. Vale conhecer.

1.2

JavaScript essencial

Você não precisa virar desenvolvedor

Precisa ler e escrever **cinco** construções

O que NÃO vamos cobrar

- Herança
- `async` / `await`
- Módulos avançados
- TypeScript

Se você ouviu essas palavras e se assustou: pode relaxar.

1 · const e let

```
const url = 'http://localhost:3001'          // não vai mudar
let contador = 0                             // vai mudar
```

`const` por padrão. `let` só quando o valor realmente trocar.

2 · Template literal

```
const email = `usuario_${Date.now()}@qa.com`
```

Acento agudo invertido, não aspa.

`${...}` é o buraco onde entra o valor.

`Date.now()` devolve um número **diferente a cada vez**.

3 · Objeto

```
const usuario = {
  nome: 'Fulano',
  email: 'fulano@exemplo.com',
  admin: true
}

usuario.nome          // 'Fulano'
usuario['email']      // 'fulano@exemplo.com'
```

4 · Array e `forEach`

```
const cenarios = ['válido', 'inválido', 'vazio']

cenarios.forEach((cenario) => {
  console.log(cenario)
})
```

"Para cada item da lista, que eu vou chamar de `cenario`, faça isto."

5 · Arrow function

```
const somar = (a, b) => a + b
```

É o mesmo que:

```
function somar(a, b) { return a + b }
```

Você vai ver `() => { ... }` o tempo todo.

Callback

O conceito que mais confunde

Callback

```
cy.get('[data-testid="email"]').then((elemento) => {
  // só roda DEPOIS que o elemento foi encontrado
})
```

Uma função que você entrega para outra pessoa executar **depois**, quando ela terminar o trabalho dela.

O café

Você pede e diz: *"quando ficar pronto, me chama"*

Você não fica olhando a máquina

O `.then()` é o **me chama**

Duas que você só precisa reconhecer

```
const { nome, email } = usuario                    // desestruturação
const novoUsuario = { ...usuario, admin: false }   // spread
```

Desestruturação tira `nome` e `email` de dentro do objeto.

Spread (`...`) copia tudo e troca uma coisa.

Não precisa decorar. Apostila, seção 1.2.

Uma semente para amanhã

Existe uma diferença entre

```
() => { }
```

e

```
function () { }
```

que vai importar **amanhã**.

Por enquanto, só saiba que existe.

1.3

Instalação e configuração

Cinco passos

Passo	O quê	Para quê
1	Node.js	Roda o Cypress e a loja
2	Git	Baixa a loja
3	VS Code	Onde você escreve os testes
4	A loja	A aplicação que vamos testar
5	O projeto Cypress	A sua suíte

O passo a passo completo está na **apostila, seção 1.3**

Passos 1 a 3

	Site	Confira no terminal
Node.js (LTS)	`nodejs.org`	`node -v` e `npm -v`
Git	`git-scm.com`	`git --version`
VS Code	`code.visualstudio.com`	—

Instale aceitando o padrão. Depois, **feche e abra** o terminal.

Linux: Node pelo `nvm`. **Mac:** o `git --version` instala o Git. Apostila, 1.3.

Visual Studio **Code** — o ícone azul. Não é o Visual Studio.

Passo 4 · A loja na sua máquina

Num **terminal** à parte, uma linha de cada vez:

```
mkdir C:\treinamento
cd C:\treinamento
git clone https://github.com/jhonatangoncalvesYwp/serverest-treinamento.git
cd serverest-treinamento
npm install
npm start
```

Mac e Linux: `~/treinamento` no lugar de `C:\treinamento`

Abra `http://localhost:3001` · `admin.treinamento@qa.com` / `treino123`

Essa janela é a loja

Minimize. **Não feche.**

Passo 5 · O projeto Cypress

VS Code → **File** → **Open Folder** → `treinamento/cypress-treinamento`

No terminal do VS Code:

```
npm init -y
npm install cypress@15.21.1 --save-dev
npx cypress open
```

Com a versão: é a do material inteiro

Na primeira abertura: **E2E Testing**

cypress.config.js

```
module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3001',
    viewportWidth: 1280,
    defaultCommandTimeout: 10000,
    video: true,
    env: { apiUrl: 'http://localhost:3000' }
  }
})
```

`baseUrl` é mais importante do que parece

Trocar de ambiente vira **uma linha**

Se travar

Sintoma	Causa provável
`node` "não é reconhecido"	Terminal aberto antes da instalação
"Execução de scripts desabilitada"	PowerShell — veja a apostila, Passo 1
Login da loja não faz nada	A janela da loja foi fechada
Download para no meio	Proxy da empresa
Executável não abre	Antivírus
`bad option: --smoke-test`	`ELECTRON_RUN_AS_NODE` no terminal

Travou? **Avise no chat** e prepare-se para compartilhar a tela.

1.4

Seu primeiro teste

Anatomia

```
describe('Página de login', () => {
  it('deve carregar a tela de login', () => {
    cy.visit('/login')
    cy.get('[data-testid="email"]').should('be.visible')
  })
})
```

`describe` agrupa · `it` é o teste

comando faz · `.should` afirma

De onde vem essa sintaxe

O Cypress **não inventou** `describe`, `it` nem `expect`.

Vem de	O que você usa dele
Mocha	`describe` `it` `before` `beforeEach` `it.only`
Chai	`expect` `to.eq` `to.contain` `to.be.an`
jQuery	O motor por trás do `cy.get()`

O que é cada um

Mocha — **organiza e executa** os testes.
Dá a estrutura e diz o que passou e o que falhou.

Chai — o **vocabulário da verificação**.
`expect(total).to.eq(3)`: *"espero que o total seja igual a 3"*.

jQuery — biblioteca antiga para **achar elementos** na página.
O `cy.get()` usa por baixo; é o que chega no `.then()`.

Pesquise `describe javascript`

Você vai cair na documentação do **Mocha**

Não está no lugar errado

O que o Cypress acrescentou

```
expect(valor).to.eq('x')                    // Chai: roda uma vez

cy.get('.alerta').should('contain', 'x')    // Cypress: tenta de novo
```

O `.should()` com nova tentativa automática.

Guarde — isso volta no 1.5.

O nome do teste é parte do teste

```
it('deve carregar a tela de login', () => { ... })
```

Quando isso falhar no relatório, essa frase é **tudo** que a pessoa vai ver.

Escreva pensando nela.

Login que funciona

```
it('deve permitir login com credenciais válidas de administrador', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')
  cy.get('[data-testid="senha"]').type('treino123')
  cy.get('[data-testid="entrar"]').click()

  cy.url().should('include', '/admin/home')
  cy.contains('h1', 'Bem Vindo').should('be.visible')
})
```

⚠️ Cuidado com o que você valida depois do login

`/admin/home` → `Bem Vindo <nome>`
`/home` (loja) → `Serverest Store`

`admin.treinamento@qa.com` é administrador. Validar `Serverest Store` aqui faria o teste falhar.

Tutoriais na internet mostram o contrário.

Vamos errar de propósito

O erro

```
cy.get('[data-testid="emai"]')   // sem o "l"
```

```
Timed out retrying after 10000ms:
Expected to find element: [data-testid="emai"],
but never found it.
```

O que essa mensagem está dizendo?

Ler o erro vale mais que decorar o comando

Você vai ver essa mensagem centenas de vezes

As duas formas de rodar

`npx cypress open` — onde você escreve

Time Travel, recarregamento automático, seletor de mira

`npx cypress run` — onde roda para valer

Sem janela, mais rápido, com vídeo e print das falhas

Escolhendo o navegador

```
npx cypress run --browser chrome
npx cypress run --browser edge
npx cypress run --browser electron    # o padrão, embutido
```

- **Electron é o padrão** — por isso roda sem Chrome instalado
- O Cypress só enxerga navegadores **instalados na máquina**
- **Safari não é suportado** oficialmente (só WebKit experimental)

`npx cypress run`

É **exatamente** este comando que vai rodar no servidor no Dia 3

Não existe mágica no CI

1.5

Comandos e asserções

Comandos por finalidade

Navegar — `cy.visit()` `cy.reload()` `cy.url()` `cy.title()`

Encontrar — `cy.get()` `cy.contains()` `.find()` `.first()` `.eq()`

Interagir — `.click()` `.type()` `.clear()` `.check()` `.select()`

Afirmar — `.should()` `.and()` `expect()`

Asserções mais usadas

```
.should('be.visible')
.should('exist')
.should('not.exist')
.should('contain', 'Bem Vindo')
.should('have.value', 'admin.treinamento@qa.com')
.should('have.length', 3)
.should('be.disabled')
```

Tabela completa no apêndice B da apostila.

Encadeamento

```
cy.get('.lista-produtos')      // acha o elemento
  .find('li')                  // acha os filhos dele
  .should('have.length', 3)    // afirma sobre os filhos
```

Cada comando recebe o resultado do anterior.

O erro que todo iniciante comete

Isto não funciona

```
cy.get('[data-testid="email"]').type('admin.treinamento@qa.com')

const email = cy.get('[data-testid="email"]').invoke('val')
expect(email).to.eq('admin.treinamento@qa.com')
```

O que vocês acham que acontece?

Falha

```
expected { Object (specWindow, chainerId) }
to equal 'admin.treinamento@qa.com'
```

Comandos do Cypress não devolvem valores.

Eles entram numa fila para serem executados depois. `email` é um objeto interno, não o texto.

O jeito certo

```
// Se quer afirmar, afirme direto
cy.get('[data-testid="email"]').should('have.value', 'admin.treinamento@qa.com')

// Se precisa do valor, use o callback
cy.get('[data-testid="email"]')
  .invoke('val')
  .then((valor) => {
    expect(valor).to.eq('admin.treinamento@qa.com')
  })
```

É o café de novo

O valor não existe na hora em que você escreve a linha

Ele existe quando ficar pronto

`should` ou `expect`?

	Quando
`.should()`	Na tela. **Tenta de novo** até passar
`expect()`	Dentro de `.then()`. Roda **uma vez só**

Usar `expect` onde caberia `should` é causa comum de teste instável.

Retry-ability

```
// ❌ ANTIPADRÃO
cy.wait(3000)
cy.get('.mensagem').should('be.visible')

// ✅ CORRETO
cy.get('.mensagem').should('be.visible')

// ✅ Para um caso que demora mais
cy.get('.relatorio', { timeout: 20000 }).should('be.visible')
```

Por que `cy.wait(3000)` é ruim

Se a tela responde em 200ms, você desperdiçou 2,8 segundos.
Vezes trinta testes.

Se num dia ruim ela demora 3,5s, seu teste falha sem motivo real.

Que número está certo?

Não existe número certo

1.6

Hooks e independência

O que é um hook

Hook = **gancho**.

Uma função que **você não chama**. Você pendura o código no gancho, e o Mocha o executa sozinho, num momento fixo.

O checklist do piloto antes de **cada** decolagem:
ninguém precisa lembrar, faz parte do procedimento.

Os quatro hooks

```
describe('Cadastro de usuário', () => {
  before(()     => { /* uma vez, antes de todos */ })
  beforeEach(() => { /* antes de CADA teste */ })
  afterEach(()  => { /* depois de cada teste */ })
  after(()      => { /* uma vez, no final */ })
})
```

Na prática, `beforeEach` **é 90% do uso**.

Vamos ver o problema acontecer

Dois testes

```
let email
it('deve cadastrar um usuário', () => {
  email = `dor_${Date.now()}@qa.com`
  // ... cadastra pela tela com esse email
})

it('deve fazer login com o usuário cadastrado', () => {
  cy.visit('/login')
  cy.get('[data-testid="email"]').type(email)
  // ... senha, entrar, valida a loja
})
```

Rode os dois: **passam**.

Agora só o segundo

```
it.only('deve fazer login com o usuário cadastrado', () => { ... })
```

```
cy.type() can only accept a string or number.
You passed in: undefined
```

O segundo teste nunca soube de onde vinha o e-mail. Estava pegando carona.

Cada `it` roda sozinho

Em qualquer ordem, quantas vezes for

Por que isso importa tanto

Alguém roda só aquele teste para investigar um bug. Ou o primeiro falha e derruba o segundo junto.

E aí o relatório acusa o **teste errado**.

Você vai passar duas horas investigando "fazer login", que está perfeito.

O Cypress já protege metade

Desde a versão 12, ele limpa cookies e `localStorage` entre os testes: a `testIsolation`.

Herdar o login do teste anterior já falha na hora.

O que ele **não** limpa: variáveis do arquivo e dados no servidor. Essa parte é com você.

A versão independente

```
it('deve fazer login com um usuário criado pelo próprio teste', () => {
  const email = `independente_${Date.now()}@qa.com`

  cy.visit('/cadastrarusuarios')
  // ... cadastra com esse email

  cy.visit('/login')
  cy.get('[data-testid="email"]').type(email)
  // ... senha, entrar, valida a loja
})
```

Cada teste prepara o que precisa. Amanhã: pela API, em milissegundos.

O teste do teste

Rode sua suíte **duas vezes seguidas**

1.7

Seletores e debugging

Hierarquia de seletores

	Tipo	Exemplo	
1	Atributo de teste	`[data-testid="entrar"]`	✅
2	`id`	`#email`	✅
3	Atributo semântico	`[name="senha"]`	✅
4	Texto visível	`cy.contains('Entrar')`	⚠️
5	Classe CSS	`.btn-primary`	⚠️
6	Caminho estrutural	`div > div > span:nth-child(3)`	🔴

O argumento

Classe CSS é decisão de estilo.
Muda quando alguém troca o tema. Ninguém avisa o QA.

Caminho estrutural quebra com uma `div` a mais.

Atributo de teste existe só para você.
Quem remove sabe que está mexendo em teste.

Se a aplicação não tem `data-testid`

Peça.

Barato para quem desenvolve, transformador para a suíte.

> *"Se vocês colocarem um atributo nesses cinco elementos, eu paro de abrir chamado toda vez que o CSS muda."*

Negociação, não reclamação.

Quando não há escolha

As tabelas do ServeRest não têm `data-testid` nenhum.

```
cy.contains('table.table-striped tbody tr', 'Nome do Produto')
  .contains('button', 'Excluir')
  .click()
```

Não é *"o botão da terceira linha"*.
É *"o botão daquela linha que fala do meu produto"*.

O alerta de mensagem também não tem: por isso `.alert`.

Como ler uma falha

O conteúdo mais útil do dia

Método de 4 passos

1. Leia a **primeira linha** do erro
2. Olhe o **screenshot** da falha
3. Volte um comando no **Time Travel**
4. Confira o seletor no **DevTools**

```
document.querySelectorAll('[data-testid="email"]')
```

Não é sobre ser bom em programação

É sobre ter um roteiro quando a tela fica vermelha

Erros comuns

Mensagem	Causa provável
`Expected to find element`	Seletor errado, ou ainda não renderizou
`element is detached from the DOM`	A página se redesenhou
`element is being covered`	Modal ou overlay por cima
`cy.visit() failed trying to load`	URL errada, app fora do ar
`Cannot read properties of undefined`	Objeto vazio no teste

1.8

Lab

Lab do Dia 1 · 40 min

Crie o arquivo `cypress/e2e/dia1/lab-dia1.cy.js` cobrindo:

1. Login com credenciais válidas de administrador
2. Login com senha inválida, validando a mensagem
3. Cadastro de novo usuário com e-mail único
4. Cadastro com e-mail já existente
5. Logout

Regras

- `beforeEach` para o que se repete
- Prefira seletores da camada 1 ou 2 (a mensagem é `.alert`)
- Cada teste roda isoladamente (teste com `it.only`)
- Nenhum `cy.wait` com número

```
const email = `usuario_${Date.now()}@qa.com`
```

Admin: `admin.treinamento@qa.com` / `treino123`

⚠️ Na tela de **cadastro** o campo de senha é `[data-testid="password"]`, não `senha`

🔴 E-mail já existente: cadastre um e-mail **seu** e tente de novo.
Nunca o do admin.

Por que e-mail único?

A sua loja guarda os dados de uma execução para a outra: `teste@qa.com` passa na primeira vez e falha na segunda

Teste que depende de dado fixo
já nasce quebrado

O que você leva do Dia 1

1. Automatize o que **dói se quebrar**, não tudo
2. Comandos **não devolvem valores** — o valor chega no `.then()`
3. Espere **condição**, nunca tempo
4. Cada teste **roda sozinho**
5. **Seletor dedicado a teste** é a primeira escolha
6. **Leia o erro** antes de mexer no código

Amanhã

Testar a API direto
Simular respostas do servidor
Parar de repetir código